SkillNavora Academy · Master Freelancing

Development and IT

Ten professions, from absolute zero to freelance-ready

Full-stack · WordPress · Web · Shopify · Mobile app · Webflow · Front-end · E-commerce · React.js · Bubble.io

Department 1 of 7 · Phase 2 draft · Official site: www.skillnavora.in

Links to official sites were not checked live in this draft. Confirm prices and links before publication.

Contents

1. How to use this textbook
2. Computer and internet fundamentals
3. Programming foundations: HTML, CSS, JavaScript, git and APIs
4. Full-stack developer
5. WordPress developer
6. Web developer
7. Shopify developer
8. Mobile app developer
9. Webflow developer
10. Front-end developer
11. E-commerce developer
12. React.js developer
13. Bubble.io developer
14. Advanced development: back end, security, performance, testing and deployment
15. Freelancing for developers: packages, discovery, proposals, pricing and delivery
16. Resource directory (Development and IT)

START HERE

How to use this textbook

This textbook teaches Development and IT from absolute zero to freelance-ready. Each profession has its own pathway, and every pathway uses the same foundations in Part 1.

Who this is for

School students (with a parent or teacher), college students, fresh graduates, working professionals, homemakers, career changers and independent freelancers. Younger learners should always work with supervision and must never open client accounts or handle payments alone.

How each lesson works

- **Simple explanation** first, then an everyday analogy.
- **Professional explanation** and the terms you will hear on the job.
- **Tools**, with free and paid options and the official link.
- **Worked example** you can copy and run.
- **Assignment** with a full solution, so you can check your own work.

Verification note

READ THIS BEFORE YOU RELY ON ANY LINK OR PRICE

Official websites, prices and plan names change often. Links in this book point to official sites, but they were **not checked live** while this book was written. Check each price and feature on the official page before you buy anything. The Resources chapter lists every link with its status.

PART 1 · MODULE A

Computer and internet fundamentals

Before you build anything for the web, you need a clear picture of what happens when someone opens a page.

A1. The computer in simple words

Simple explanation. A computer stores information, calculates, and shows results. An operating system (Windows, macOS, Android, iOS or Linux) manages the hardware so programs can use it.

Everyday analogy. A school: the storeroom keeps files (storage), teachers do the work (processor), the notice board shows results (screen). The principal who keeps everything running is the operating system.

A2. The internet and the browser

Simple explanation. The internet is a network of computers that send messages to each other. A browser (Chrome, Edge, Firefox, Safari) asks a computer for a web page and displays it.

Everyday analogy. Ordering food online: you pick a shop (website), the kitchen prepares the order (server), and a delivery brings it to your door (the internet).

A3. Websites, web apps, domains and hosting

Term	Simple meaning	Example
Website	Pages that show information	A bakery's menu
Web application	A website where you can do something (log in, save data, pay)	A booking form that saves appointments
Domain	The address people type, such as `example.com`	The shop's name on the street
Hosting	A computer that stays on and serves your site	The shop building you rent

A4. Servers, databases and client–server architecture

Simple explanation. The **client** (usually a browser) asks for something. The **server** answers. A **database** is an organised store, like a school register with one row per student.

TEXT

```
Client (browser) --- request: GET /products ---> Server
Client (browser) <--- response: product list ---- Server
                                                    |
                                                    v
                                        Database (products table)
```

A5. HTTP and HTTPS

HTTP is the language browsers and servers use. A request has a **method** (GET to read, POST to send data), an **address** and **headers**. The reply has a **status code**: 200 means success, 404 means not found, 500 means the server failed. HTTPS is HTTP with encryption. Every public site you build must use HTTPS.

A6. Basic cybersecurity for beginners

- Use a password manager and two-factor authentication on every work account.
- Never paste secret keys into a website, a chat or a public repository.
- Be sceptical of unexpected invoices, login pages and urgent payment requests.

ASSIGNMENT

Map a page request

Business problem: A client wants staff to understand what happens when a customer opens the shop's website. Draw the flow.

Steps

1. Write the steps from typing the address to seeing the page.
2. Label each step as client, server, network or database.
3. Mark the step where HTTPS protects the data.

Expected output: A numbered list of six to eight labelled steps.

SOLUTION (ANSWER KEY)

1. The customer types `shop.example.com` in the browser (client).
2. The browser asks the domain name system for the server's address (network).
3. The browser sends `GET /` over HTTPS (client to server, encrypted).
4. The server asks the database for the products (server to database).
5. The database returns the rows; the server builds an HTML page and replies with status 200 (server).
6. The browser displays the page (client).

Why it works: the steps follow the client–server model. Step 3 is where encryption protects the request on its way.

PART 1 · MODULE B

Programming foundations: HTML, CSS, JavaScript, git and APIs

Every web profession starts with these languages and tools. You do not need to master them before you begin; you need to build and fix a small page.

B1. HTML: the structure

Simple explanation. HTML is the skeleton of a page. Headings, paragraphs, images, buttons and forms are HTML elements.

Everyday analogy. HTML is the layout of a printed menu: where the title goes, which items form a list, where the prices sit.

HTML

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
<title>Asha's Bakery</title></head>
<body>
  <h1>Asha's Bakery</h1>
  <p>Fresh bread every morning.</p>
  <ul><li>Sourdough</li><li>Croissant</li></ul>
  <a href="mailto:hello@example.com">Order by email</a>
</body></html>
```

B2. CSS: the look

Simple explanation. CSS controls colour, size, spacing and layout. A rule selects elements and sets properties.

CSS

```
body { font-family: system-ui, sans-serif; margin: 0; padding: 16px; color: #1d1640; }
h1 { color: #6c4cf5; }
@media (min-width: 720px) { body { padding: 40px; max-width: 720px; margin: auto; } }
```

B3. Responsive design

A responsive page works on a phone (about 390 pixels wide) and a laptop (about 1280 pixels). Use the viewport meta tag, flexible widths (`max-width: 100%`) and media queries. Test both widths before delivery.

B4. JavaScript: behaviour

Simple explanation. JavaScript makes a page react: clicks, form checks and messages. It is a set of instructions the browser runs.

JAVASCRIPT

```
const form = document.querySelector('#enquiry');
const msg = document.querySelector('#message');
form.addEventListener('submit', (event) => {
  event.preventDefault();                        // stop the page reloading
  const digits = form.phone.value.replace(/\D/g, '');
  msg.textContent = digits.length < 10
```

```
    ? 'Please enter a 10-digit phone number.'
    : 'Thank you. We will call you soon.';
});
```

B5. Programming logic

Programs follow three ideas: **sequence** (steps in order), **decision** (if this, then that) and **repetition** (do this for each item). Write the logic in plain sentences before you write code.

B6. Git and GitHub

Simple explanation. Git records every change, so you can go back. GitHub stores a copy online so others can see and share it.

BASH

```
git init                      # start tracking this folder
git add index.html            # choose the files to save
git commit -m "Add bakery homepage"
git log --oneline             # see your history
```

B7. Command-line basics

The terminal runs commands instead of clicks. Useful ones: `pwd` (where am I), `ls` (list files), `cd folder` (move into a folder), `mkdir folder` (create a folder).

B8. Debugging

1. Read the error message exactly. It usually names the file and the line.
2. Reproduce the problem with the smallest example that shows it.
3. Use the browser developer tools (right-click, Inspect, Console).
4. Change one thing at a time, then test again.

B9. APIs

Simple explanation. An API is a messenger that lets two programs talk. You send a request in an agreed format and receive a structured reply, usually JSON.

JSON

```
{
  "name": "Sourdough",
  "price_inr": 180,
  "in_stock": true
}
```

B10. Basic databases

A table has columns (fields) and rows (records). A key identifies each row. SQL is the language for reading and writing tables.

SQL

```
CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price_inr INTEGER);
INSERT INTO products VALUES (1, 'Sourdough', 180);
SELECT name, price_inr FROM products WHERE price_inr < 250;
```

OFFICIAL LEARNING LINKS (NOT CHECKED LIVE IN THIS SESSION)

- MDN Web Docs: Learn web development
- Pro Git (free book)
- GitHub Skills

ASSIGNMENT

Build and publish a one-page bakery site

Business problem: A bakery needs a simple online menu and an order form that checks the phone number before it sends.

Steps

1. Create `index.html` with a heading, a three-item menu list and a form with name, phone and message fields.
2. Create `style.css` so the page is readable on a phone and a laptop.
3. Create `script.js` that stops the form reloading and checks for at least 10 digits in the phone field.
4. Save the work with git and make at least two commits.

Expected output: A folder with three files and a git history of at least two commits. A short phone number shows an error; a valid one shows the thank-you message.

SOLUTION (ANSWER KEY)

HTML

```
<!-- index.html -->
<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Asha's Bakery</title><link rel="stylesheet" href="style.css"></head>
<body>
<h1>Asha's Bakery</h1>
<ul><li>Sourdough - Rs 180</li><li>Croissant - Rs 90</li><li>Rye loaf - Rs 220</li></ul>
<form id="enquiry">
  <label>Name <input name="name" required></label>
  <label>Phone <input name="phone" required></label>
  <label>Message <textarea name="message"></textarea></label>
  <button>Order</button>
</form>
<p id="message" role="status"></p>
<script src="script.js"></script>
</body></html>
```

CSS

```
*, *::before, *::after { box-sizing: border-box; }
body { font-family: system-ui, sans-serif; margin: 16px; color: #1d1640; }
h1 { color: #6c4cf5; }
label { display: block; margin: 8px 0; }
input, textarea { width: 100%; max-width: 360px; padding: 8px; }
@media (min-width: 720px) { body { max-width: 720px; margin: 40px auto; } }
```

JAVASCRIPT

```
// script.js
const form = document.querySelector('#enquiry');
const msg = document.querySelector('#message');
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const digits = form.phone.value.replace(/\D/g, '');
  msg.textContent = digits.length < 10
    ? 'Please enter a 10-digit phone number.'
    : 'Thank you. We will call you soon.';
});
```

Why it works: HTML gives the structure, CSS adapts the layout to the screen width, and the script stops the browser reloading the form. The digit check removes spaces and symbols first, so '+91 98765-43210' counts as 12 digits and passes.

DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 1

Full-stack developer

Profession 1 of 10 in Development and IT.

What the profession means

A full-stack developer builds both sides of a web application: the screens people see (front end) and the server, database and logic behind them (back end).

What professionals do every day

Plans features, builds screens, writes API endpoints, designs tables, fixes bugs, writes tests and deploys the application. Works with git, reads error logs and talks to clients about what they need.

Why businesses hire them, and what problems they solve

A business with a booking, shop or dashboard needs one person who can carry a feature from the database to the button. Full-stack developers reduce hand-offs and can deliver a complete product.

- Turning a client's idea into screens, data and rules.
- Keeping data safe and consistent between the screen and the database.
- Shipping a working version quickly, then improving it.

Skills you need

- HTML, CSS and JavaScript (Part 1, Module B).
- A front-end framework such as React (see Profession 9).
- Node.js and an API framework.
- Databases (SQL and at least one hosted option).
- Authentication, validation and error handling.
- Git, testing and deployment.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Visual Studio Code	Code editor	Free	Free	code.visualstudio.com
Node.js	Runs JavaScript on the server	Free	Free	nodejs.org
GitHub	Code hosting and collaboration	Free tier	Paid team plans	github.com
Supabase	Hosted Postgres database and authentication	Free tier	Paid plans	supabase.com
Vercel	Hosting for front-end and serverless back-end	Hobby tier	Paid plans	vercel.com

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Weeks 1–4: HTML, CSS, JavaScript, git. Build the bakery site from Part 1.
Intermediate	Weeks 5–10: React basics, Node API, one database table, form to database.
Advanced	Weeks 11–20: authentication, tests, deployment, a client-style feature with a real data flow.

Projects by level

- **Beginner:** A notes app with add, list and delete, stored in a file or database.
- **Intermediate:** A booking form that saves to a database and sends an email confirmation.
- **Advanced:** A multi-user dashboard with sign-in, roles and an admin view.

Client-ready portfolio project

A complete application for a small business: sign-in, a list of records, a form, a database, deployment on a public address and a README that explains how to run it. Use a real business (with permission) or a clearly labelled fictional one.

Worked example

Worked example. A clinic wants a 'book a visit' page. The chain is: form (front end) sends a POST to `/api/bookings` (back end), the server validates the date and phone number, the database stores the row, and the page shows a confirmation. Each part can be tested alone.

Assignment with full solution

ASSIGNMENT

A notes API with validation

Business problem: A small team needs a notes service for its website. The API must list notes, add a note, and reject empty text.

Steps

1. Create `server.js` as shown in the solution. Start it with `node server.js`.
2. Test with a browser at `/api/notes` (GET).
3. Send a POST with `{"text": "Buy milk"}` and check for status 201.
4. Send a POST with empty text and check for status 400.

Expected output: A GET returns a JSON list; a valid POST returns 201 with the new note; an empty text returns 400.

SOLUTION (ANSWER KEY)

The solution uses only Node's built-in `http` module, so nothing needs installing. Each rule maps to a status code: 200 for reading, 201 for creating, 400 for bad input, 404 for unknown routes. Tested with `curl -X POST -d '{"text":""}'`, which returns 400.

JAVASCRIPT

```
// server.js  -- a tiny JSON API using only Node's built-in http module
const http = require('http');
let notes = [{ id: 1, text: 'Call supplier' }];
let nextId = 2;
const send = (res, status, body) => {
  res.writeHead(status, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(body));
};
http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/api/notes') return send(res, 200, notes);
  if (req.method === 'POST' && req.url === '/api/notes') {
    let raw = '';
    req.on('data', (c) => (raw += c));
    req.on('end', () => {
      let data;
      try { data = JSON.parse(raw); } catch { return send(res, 400, { error: 'invalid JSON'
}); }
      if (!data.text || !data.text.trim()) return send(res, 400, { error: 'text is required'
});
      const note = { id: nextId++, text: data.text.trim() };
      notes.push(note);
      send(res, 201, note);
    });
    return;
  }
  send(res, 404, { error: 'not found' });
}).listen(3000, () => console.log('API on http://localhost:3000'));
```

Common mistakes

- Writing the database and API before agreeing the screens with the client.
- Storing passwords in plain text. Always hash them and use an authentication service.
- Skipping validation on the server, which trusts only the browser.
- Deploying with secret keys in the code.

Freelancing opportunities

- Freelance web application builds for small businesses: booking systems, portals, dashboards.
- Maintenance and feature retainers after launch.
- Starting points on freelance marketplaces (see Freelancing chapter): search for 'full stack', 'React', 'Node', 'API' jobs and apply with a working demo.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- MDN: Learn web development
- Node.js learning resources
- The Odin Project (free full-stack curriculum)
- Supabase documentation

DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 2

WordPress developer

Profession 2 of 10 in Development and IT.

What the profession means

A WordPress developer builds and customises websites on WordPress, the open-source content management system that powers a large share of the web. The work ranges from setting up a site with a theme to writing custom plugins and child themes.

What professionals do every day

Installs and configures WordPress, chooses and customises themes, adds plugins, creates content types, sets up hosting and security, and maintains sites with backups and updates.

Why businesses hire them, and what problems they solve

Most small businesses want a site they can edit themselves. WordPress gives them that, and a developer makes it fast, secure, and designed for their customers.

- Sites that are hard to edit or update.
- Slow pages and weak security from poorly chosen plugins.
- Custom features such as booking, courses or product catalogues.

Skills you need

- How WordPress works: pages, posts, themes, plugins and the database.
- HTML, CSS and some PHP for custom code.
- Child themes and safe customisation.
- Hosting, backups, security and performance.
- SEO basics (see Marketing department).

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
WordPress.org	Open-source software to install yourself	Free	Hosting costs only	wordpress.org
WordPress.com	Hosted WordPress service	Free plan	Paid plans	wordpress.com
Local	Run WordPress on your own computer for testing	Free	Paid tiers	localwp.com
Wordfence / security plugins	Security scanning and firewall	Free version	Paid plans	www.wordfence.com
UpdraftPlus	Backups	Free version	Paid plans	updraftplus.com

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Install WordPress locally; build pages with the block editor; learn themes and plugins.
Intermediate	Create child themes; edit templates; add custom CSS and simple PHP snippets with care.
Advanced	Build custom post types, performance tuning, security hardening and a staging workflow.

Projects by level

- **Beginner:** A five-page business site with a contact form and a backup schedule.
- **Intermediate:** A custom booking page using a plugin and a child theme.
- **Advanced:** A membership or course site with user roles and payments.

Client-ready portfolio project

A complete business site with a child theme, staging copy, backups, performance and security settings, and a handover document for the client.

Worked example

Worked example. Changing a button colour is done in the child theme's `style.css`, never the parent theme. The parent can then update without losing the change.

Assignment with full solution

ASSIGNMENT

Build a five-page bakery site on WordPress with a child theme

Business problem: A bakery needs a home, menu, about, contact and order page, a child theme for custom styling, and a plan for updates.

Steps

1. Install WordPress on a local or hosted test site (see official install guide).
2. Choose a block theme and activate it. Create the pages Home, Menu, About, Contact and Order.
3. Create a child theme folder named `bakery-child` with the `style.css` shown, and activate it.
4. Add a hero section with class `bakery-hero` to the Home page.
5. Install one backup plugin and one security plugin; record the settings used.

Expected output: Five published pages, a working child theme, the hero styling visible on the home page, and a short maintenance note.

SOLUTION (ANSWER KEY)

The child theme keeps your changes separate, so theme updates do not erase them. The header's *Template* line must match the folder name of the installed parent theme (here twentytwentyfour, the default block theme; check your installed version). Activating the child theme and seeing the hero

background proves it works. Maintenance note: schedule weekly backups, update plugins after testing on a copy, and keep administrator accounts to two people.

CSS

```
/*
Theme Name: Bakery Child
Template: twentytwentyfour
Description: Child theme for the bakery site. Keeps customisations safe during updates.
Version: 1.0
*/
/* Custom styles go below. The parent theme name must match an installed theme. */
.bakery-hero { background: #fff7e8; padding: 40px 16px; text-align: center; }
```

Common mistakes

- Editing the parent theme directly, which is lost on update.
- Installing many plugins; each one adds risk and slowness.
- Working on a live site without a staging copy or a backup.
- Using 'nulled' (pirated) themes or plugins, which can contain malware.

Freelancing opportunities

- Small business websites, landing pages and maintenance plans.
- Fixing and speeding up existing sites.
- Freelance marketplace projects: search 'WordPress', 'Elementor', 'WooCommerce'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- WordPress official documentation
- WordPress developer handbook: child themes
- WordPress Learn (official courses)

Web developer

Profession 3 of 10 in Development and IT.

What the profession means

A web developer builds websites: the pages, layout and interactivity that visitors use. A web developer usually works with HTML, CSS, JavaScript and sometimes a content system or framework.

What professionals do every day

Turns designs into working pages, makes them responsive and accessible, connects forms and analytics, tests across browsers and devices, and publishes the site.

Why businesses hire them, and what problems they solve

Customers judge a business by its website in seconds. A developer makes the site fast, readable on phones, easy to contact the business through, and found by search engines.

- Designs that look good but break on a phone.
- Pages that are slow or hard to use for people with disabilities.
- Forms that lose enquiries.

Skills you need

- HTML semantics, CSS layout (flexbox and grid), responsive design.
- JavaScript for interaction and form checks.
- Accessibility basics (labels, contrast, keyboard use).
- Performance basics (image size, caching).
- Publishing and hosting.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Visual Studio Code	Code editor	Free	Free	code.visualstudio.com
Chrome DevTools	Inspect, debug and test layouts	Free	Free	developer.chrome.com/docs/devtools
Netlify	Publish static sites	Free tier	Paid plans	www.netlify.com
GitHub Pages	Free hosting for static sites from a repository	Free	Free	pages.github.com
W3C validator	Checks HTML errors	Free	Free	validator.w3.org

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Static pages with HTML and CSS; responsive layouts; a simple form with checks.
Intermediate	JavaScript interactions, accessible components, forms that save to a service.
Advanced	Performance tuning, a component library, analytics and an automated test.

Projects by level

- **Beginner:** A four-page site for a local service business.
- **Intermediate:** A booking page with a calendar and a working contact form.
- **Advanced:** A content-heavy site with search, filters and measured performance.

Client-ready portfolio project

A responsive, accessible, search-friendly site for a real or clearly fictional local business, with a performance report and a handover page.

Worked example

Worked example. Turning a design card into HTML: a card is a `<article>` with a heading, a paragraph and a link. CSS gives it padding and a border; the grid places the cards.

Assignment with full solution

ASSIGNMENT

A responsive business landing page

Business problem: A plumbing company needs a single landing page with services, prices and a call button that works on a phone.

Steps

1. Build a header with the company name, a 'Call now' link using `tel:`, and a menu of three services.
2. Add a services section with three cards that stack on a phone and sit in a row on a laptop.
3. Add a contact form with name, phone and service fields; label every input.
4. Test at 390 and 1280 pixels and fix any horizontal scrolling.

Expected output: One HTML page, one CSS file, accessible labels, and no horizontal scrolling at 390 pixels.

SOLUTION (ANSWER KEY)

Solution outline: the call link uses `<a href="tel:+919800000000">`, which opens the phone dialler on a mobile device. The cards use CSS grid with `grid-template-columns: 1fr` by default and `repeat(3, 1fr)` inside a media query for widths of 720 pixels and above. Each input has a visible <label>, which screen readers and autofill rely on.

CSS

```
.cards { display: grid; gap: 16px; grid-template-columns: 1fr; }
@media (min-width: 720px) { .cards { grid-template-columns: repeat(3, 1fr); } }
a.call { display: inline-block; padding: 12px 18px; background: #6c4cf5; color: #fff; border-radius: 10px; text-decoration: none; }
```

Test every page at 390 and 1280 pixels, check that all images have alt text, and validate the HTML with the free W3C validator (validator.w3.org).

Common mistakes

- Using `<div>` for everything instead of meaningful elements.
- Forgetting to test on a phone until the end.
- Unoptimised images that slow the page.
- Forms without labels or error messages.

Freelancing opportunities

- Small business landing pages and brochure sites.
- Fixing and redesigning existing websites.
- Freelance marketplace briefs for 'landing page', 'HTML/CSS', 'responsive website'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- web.dev Learn (Google)
- MDN: Learn web development
- W3C Web Accessibility tutorials

Shopify developer

Profession 4 of 10 in Development and IT.

What the profession means

A Shopify developer builds and customises online shops on Shopify, a hosted e-commerce platform. The work covers themes (built with Liquid), apps, product data, checkout settings and integrations with payments, shipping and accounting.

What professionals do every day

Customises theme sections, sets up products and collections, installs and configures apps, connects payments and shipping, tests checkout, and fixes storefront issues.

Why businesses hire them, and what problems they solve

Merchants want to sell without running servers. A Shopify developer makes the store look right, load fast, and handle the business rules (discounts, bundles, delivery areas) that the basic theme cannot.

- Stores that look generic or load slowly.
- Manual order and stock work that an integration can automate.
- Checkout and shipping rules that a standard setup does not cover.

Skills you need

- E-commerce basics: products, variants, collections, orders, taxes.
- HTML, CSS and JavaScript; Liquid templating.
- Shopify admin, theme editor and apps.
- Performance and accessibility of storefronts.
- Basic integrations (APIs and webhooks, see Part 1 Module B).

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Shopify	E-commerce platform	Free trial and development stores through the official programme	Paid plans	www.shopify.com
Shopify Dev Docs	Theme and app developer reference	Free	Free	shopify.dev
Shopify CLI	Command-line tool for theme development	Free	Free	shopify.dev/docs/apps/tools/cli
Shopify Partners	Partner programme with development stores	Free to join	Free to join	www.shopify.com/partners

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish

Beginner	Admin setup, products, collections, discounts; edit an existing theme's text and images.
Intermediate	Theme sections with Liquid and schema; install and configure one app; test checkout in a development store.
Advanced	Custom app integrations with the Admin API, webhooks, performance audits and multi-currency or multi-region setup.

Projects by level

- **Beginner:** A store for one product category with a custom homepage section.
- **Intermediate:** A bundle or discount rule set up with an app and tested.
- **Advanced:** An integration that syncs orders to a spreadsheet or CRM.

Client-ready portfolio project

A complete store for a fictional or permissioned business: theme customisation, product catalogue, shipping and tax set up according to official guidance, and a checkout test log.

Worked example

Worked example. Adding a 'Free delivery over Rs 500' banner: create a text section in the theme editor, write the condition in a shipping rule, and test one cart above and one below the threshold.

Assignment with full solution

ASSIGNMENT

A featured-products section for a bakery store

Business problem: The bakery owner wants a homepage section that shows three bestsellers from a chosen collection, with a heading the owner can edit.

Steps

1. Create a development store from the Shopify Partners programme or a trial account (official sign-up page).
2. Add a collection named 'Bestsellers' and three products with images and prices.
3. Create the file `sections/featured-products.liquid` as in the solution.
4. Add the section to the homepage in the theme editor and set the heading and collection.
5. Check the page on a phone and a laptop.

Expected output: A homepage section that shows three products with image, title and price, editable in the theme editor.

SOLUTION (ANSWER KEY)

The Liquid loop reads the products from the chosen collection and limits the output to three. The schema block tells the theme editor which settings to show, so the owner can change the heading and collection without touching code. `loading="lazy"` delays off-screen images, which helps speed. **Not run here:** this needs a Shopify store, which was not reachable from this session; check the current Liquid and schema reference in the Shopify developer documentation before publishing.

LIQUID

```liquid
{%- comment -%} sections/featured-products.liquid (simplified) {%- endcomment -%}
<section class="featured">
  <h2>{{ section.settings.heading }}</h2>
  <div class="grid">
    {%- for product in collections[section.settings.collection].products limit: 3 -%}
      <article class="card">
        <a href="{{ product.url }}">
          <img src="{{ product.featured_image | image_url: width: 600 }}" alt="{{
product.featured_image.alt | escape }}" loading="lazy">
          <h3>{{ product.title }}</h3>
        </a>
        <p>{{ product.price | money }}</p>
      </article>
    {%- endfor -%}
  </div>
</section>
{% schema %}
{ "name": "Featured products", "settings": [
  { "type": "text", "id": "heading", "label": "Heading", "default": "Our bestsellers" },
  { "type": "collection", "id": "collection", "label": "Collection" } ],
  "presets": [{ "name": "Featured products" }] }
{% endschema %}
```

Common mistakes

- Editing the live store before testing in a development store.
- Installing many apps that slow the store and conflict with each other.
- Forgetting to set tax and shipping rules for each region.
- Copying third-party themes without a licence.

Freelancing opportunities

- Store setup and redesign for small brands.
- Monthly maintenance (app updates, new products, banners).
- Integrations for order and stock sync; search 'Shopify', 'Liquid', 'Shopify app' on freelance marketplaces.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- Shopify theme development docs
- Shopify API documentation
- Shopify Help Centre

Mobile app developer

Profession 5 of 10 in Development and IT.

What the profession means

A mobile app developer builds applications for phones and tablets. Most work uses one code base for both Android and iPhone through a cross-platform tool such as React Native (with Expo) or Flutter, or native tools (Kotlin for Android, Swift for iPhone).

What professionals do every day

Designs screens for touch, connects to data and APIs, handles notifications and permissions, tests on real devices, and prepares store submissions.

Why businesses hire them, and what problems they solve

Customers spend more time on phones than on desktops. Businesses want an app for bookings, loyalty, ordering or internal work, and a developer can deliver one code base for both stores.

- Apps that feel slow, crash or drain the battery.
- Login, payments and notifications that must work on both platforms.
- Store rejections caused by missing policies and poor testing.

Skills you need

- JavaScript or Dart (depending on the tool), and component thinking.
- Navigation, forms, lists and state.
- APIs, storage and authentication.
- Device testing, permissions and performance.
- App store rules and privacy policies.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Expo	Build React Native apps with a simple workflow	Free	Paid build services	expo.dev
React Native	Cross-platform UI framework	Free	Free	reactnative.dev
Flutter	Cross-platform UI toolkit with Dart	Free	Free	flutter.dev
Android Studio	Official Android IDE	Free	Free	developer.android.com/studio
Xcode	Official Apple development tool (needs a Mac)	Free	Free (Apple developer account for publishing has a fee)	developer.apple.com/xcode

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Screens, lists, forms and local state with a single-screen app.
Intermediate	Navigation, data from an API, storage, and testing on a real phone.
Advanced	Authentication, push notifications, offline support, performance tuning and store submission.

Projects by level

- **Beginner:** A checklist or habit tracker.
- **Intermediate:** A booking app that reads from an API.
- **Advanced:** A loyalty app with login, notifications and an admin view.

Client-ready portfolio project

A two-screen app for a real or clearly fictional local business (ordering, bookings or loyalty), tested on Android and iPhone, with a privacy policy and store-ready screenshots.

Worked example

Worked example. A list screen is a `FlatList` fed by an array. Adding an item means creating a new array, which the list redraws. Changing an item means mapping over the array and replacing one object.

Assignment with full solution

ASSIGNMENT

A habit checklist app on a phone

Business problem: A student wants a phone app to add daily study habits and tick them off. It must reject empty names and work on both Android and iPhone.

Steps

1. Create an Expo project from the official template and open it in Expo Go on your phone.
2. Replace `App.js` with the solution.
3. Add three habits, tick one, and check that empty input is ignored.
4. Close and reopen the app; note that the data is lost (this is expected in this version).

Expected output: A working checklist on your phone: add, tick and ignore empty input. Notes on what is not yet saved.

SOLUTION (ANSWER KEY)

The solution keeps habits in memory with `useState`, so an empty name is ignored by the `trim()` check. Tapping a row flips its `done` flag using an immutable update (a new array), which React needs to redraw the list. **Not run here:** this needs Expo and a phone; the code follows the Expo template conventions, but test it on a device before you rely on it. The next step, saving data, uses a storage library that the Expo documentation covers.

JAVASCRIPT

```
// App.js (React Native with Expo). Run with: npx expo start, then scan the QR code in Expo
Go.
import { useState } from 'react';
import { Button, FlatList, StyleSheet, Text, TextInput, View } from 'react-native';

export default function App() {
  const [habits, setHabits] = useState([]);
  const [name, setName] = useState('');

  const addHabit = () => {
    const clean = name.trim();
    if (!clean) return;                       // ignore empty input
    setHabits([...habits, { id: String(Date.now()), name: clean, done: false }]);
    setName('');
  };

  const toggle = (id) =>
    setHabits(habits.map((h) => (h.id === id ? { ...h, done: !h.done } : h)));

  return (
    <View style={styles.screen}>
      <Text style={styles.title}>Study habits</Text>
      <TextInput style={styles.input} value={name} onChangeText={setName}
                 placeholder="New habit, e.g. 20 minutes of SQL" />
      <Button title="Add" onPress={addHabit} />
      <FlatList data={habits} keyExtractor={(h) => h.id}
        renderItem={({ item }) => (
          <Text style={styles.row} onPress={() => toggle(item.id)}>
            {item.done ? '✓ ' : '○ '}{item.name}
          </Text>)} />
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, padding: 24, paddingTop: 60 },
  title: { fontSize: 24, fontWeight: '700', marginBottom: 12 },
  input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 8, padding: 10, marginBottom: 8
},
  row: { fontSize: 18, paddingVertical: 10 },
});
```

Common mistakes

- Testing only in an emulator; always test on a real device.
- Ignoring the store's privacy and permission rules.
- Storing secrets (API keys) inside the app bundle.
- Building without a plan for offline or slow networks.

Freelancing opportunities

- App builds for small businesses (booking, loyalty, ordering).
- Store submission and maintenance contracts.
- Search 'React Native', 'Expo', 'Flutter', 'mobile app' on freelance marketplaces; present a working demo on a phone.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- React Native: getting started
- Expo documentation
- Flutter documentation
- Android Developers courses

DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 6

Webflow developer

Profession 6 of 10 in Development and IT.

What the profession means

A Webflow developer builds websites in Webflow, a visual no-code and low-code platform that writes clean HTML, CSS and JavaScript from the layout the developer draws. Webflow also has a content management system (CMS) for editable content.

What professionals do every day

Designs and builds pages visually, sets up responsive breakpoints, creates CMS collections, adds interactions and animations, connects forms and integrations, and publishes and manages hosting.

Why businesses hire them, and what problems they solve

Businesses that want a design-led site and an editor they can use often choose Webflow. A developer builds the structure, makes it responsive and accessible, and gives the client a site that is easy to maintain.

- Sites that are hard to update after launch.
- Designs that do not adapt to phones.
- Blogs and listings that need many similar pages.

Skills you need

- HTML and CSS: the box model, flexbox and grid.
- Webflow Designer: classes, styles, breakpoints, components.
- CMS collections and templates.
- Forms, integrations and basic interactions.
- SEO settings, accessibility and performance.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Webflow	Visual website builder with CMS	Free plan for building and webflow.io subdomains	Paid site and workspace plans	webflow.com
Webflow University	Official free lessons	Free	Free	university.webflow.com
Figma	Design before building (see Design department)	Free starter plan	Paid plans	www.figma.com

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Build a one-page site with sections, containers and responsive breakpoints.

Intermediate	Add a CMS collection and a list page; style and link them.
Advanced	Build templates, interactions, forms with integrations, and a multi-language or multi-page site.

Projects by level

- **Beginner:** A four-page brochure site for a local shop.
- **Intermediate:** A blog with a collection list and a template page.
- **Advanced:** A listing site with filters and an admin-friendly CMS.

Client-ready portfolio project

A complete site for a real or fictional business: design system with classes, responsive layout, CMS blog, forms, SEO fields and a handover video or document for the client.

Worked example

Worked example. A service card is a Div with a class `card`. Styling the class once updates every card; a variant changes only the card that needs it.

Assignment with full solution

ASSIGNMENT

A Webflow site with a CMS blog for a bakery

Business problem: The bakery wants a four-page site and a blog where the owner can publish a post without touching the design.

Steps

1. Create a Webflow account and a new site from a blank project (official Webflow University lessons are free).
2. Build Home, Menu, About and Contact pages with the Designer, using a container, a grid of three cards and a navigation bar.
3. Create a CMS Collection named 'Posts' with fields: title, date, body (rich text), cover image.
4. Add a Collection List on the blog page bound to Posts; add a Collection Template page for one post.
5. Publish to a Webflow subdomain and check the blog on a phone.

Expected output: Four static pages, a blog with a list page and a template page, and a published preview link.

SOLUTION (ANSWER KEY)

Solution outline. The layout uses a Section, a Container and a grid with three columns that collapse to one column on small screens. The Posts Collection holds the content; the Collection List repeats one card per item; the Collection Template page displays the full post. Editing a post in the CMS updates the site without a designer. **Not run here:** Webflow was not reachable from this session; check the current Designer and CMS steps in the official Webflow University before following them.

Common mistakes

- Styling each element individually instead of using classes.
- Ignoring breakpoints and checking only on a laptop.
- Putting the whole site in one long page with no structure for search engines.
- Publishing without a handover guide for the client.

Freelancing opportunities

- Brochure and portfolio sites for designers and small firms.
- Webflow CMS blogs and listings.
- Freelance briefs that ask for 'Webflow', 'no-code website', 'CMS blog'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- Webflow University
- Webflow: intro to the Designer (official lesson)
- Made in Webflow (examples)

DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 7

Front-end developer

Profession 7 of 10 in Development and IT.

What the profession means

A front-end developer builds the part of a website or app that users see and use: layout, interaction, accessibility and performance. The work happens in HTML, CSS and JavaScript, often with a framework such as React.

What professionals do every day

Turns designs into components, makes them accessible and responsive, connects screens to data from the back end, tests in several browsers and fixes performance problems.

Why businesses hire them, and what problems they solve

Users decide in seconds whether a product is trustworthy. Front-end developers make it fast, usable and inclusive, which also helps conversions and search rankings.

- Pages that do not work with a keyboard or screen reader.
- Slow sites with heavy images and scripts.
- Layouts that break on some phones or browsers.

Skills you need

- HTML semantics and forms.
- CSS layout, responsive design and animation basics.
- JavaScript for interaction, the DOM and events.
- Accessibility (WCAG basics), performance and browser testing.
- A component framework such as React (see Profession 9).

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Visual Studio Code	Code editor	Free	Free	code.visualstudio.com
Chrome DevTools	Debug layout, network and performance	Free	Free	developer.chrome.com/docs/devtools
web.dev	Official guidance on performance and accessibility (Google)	Free	Free	web.dev
axe DevTools	Accessibility checks in the browser	Free extension	Paid tiers	www.deque.com/axe/
Lighthouse	Performance and accessibility audits	Free	Free	developer.chrome.com/docs/lighthouse

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	HTML and CSS layouts that adapt; forms with clear errors.
Intermediate	Interactive components (accordions, menus, tabs) that work with the keyboard.
Advanced	Performance budgets, lazy loading, component libraries and automated accessibility tests.

Projects by level

- **Beginner:** A responsive landing page with a working FAQ.
- **Intermediate:** A product page with a gallery, filters and a cart icon.
- **Advanced:** A component library with documented accessible states.

Client-ready portfolio project

A responsive, accessible site for a real or fictional business, with a Lighthouse report over 90 in performance and accessibility on mobile (check the result on your own run), and a short accessibility statement.

Worked example

Worked example. Making a menu accessible: use `<nav>`, make each link keyboard reachable, and give the open state to assistive technology with `aria-expanded`.

Assignment with full solution

ASSIGNMENT

An accessible FAQ accordion

Business problem: A website's FAQ must open and close answers with a mouse, a touch screen and a keyboard, and tell screen readers which answer is open.

Steps

1. Use a real `<button>` for each question, not a styled div.
2. Link each button to its answer with `aria-controls`, and set `aria-expanded` to true or false.
3. Toggle the `hidden` attribute on the answer when the button is pressed.
4. Check that Tab and Enter (or Space) open and close each answer.

Expected output: Each question opens and closes its answer; the screen reader announces expanded or collapsed; the keyboard works.

SOLUTION (ANSWER KEY)

Why it works. A native button gets keyboard focus and Enter/Space handling for free. `aria-expanded` tells assistive technology the state, and `aria-controls` links the button to its panel. Removing the `hidden` attribute shows the answer. Tested in a browser: clicking each question toggles its panel and the state attribute.

HTML

```
<!-- faq.html : accessible accordion (buttons, aria, keyboard) -->
<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1"><title>FAQ</title>
<style>
  body { font-family: system-ui, sans-serif; margin: 16px; color: #1d1640; }
  .q { width: 100%; text-align: left; padding: 12px; font-size: 1rem; border: 1px solid #ccc;
background: #fff; }
  .a { padding: 12px; border: 1px solid #ccc; border-top: 0; }
  .a[hidden] { display: none; }
</style></head>
<body>
<h1>Delivery questions</h1>
<h3><button class="q" aria-expanded="false" aria-controls="a1" id="b1">Do you deliver on
Sundays?</button></h3>
<div class="a" id="a1" role="region" aria-labelledby="b1" hidden><p>Yes, from 8 am to 6 pm.
</p></div>
<h3><button class="q" aria-expanded="false" aria-controls="a2" id="b2">What is the minimum
order?</button></h3>
<div class="a" id="a2" role="region" aria-labelledby="b2" hidden><p>Rs 300 across all items.
</p></div>
<script>
  document.querySelectorAll('.q').forEach((btn) => {
    btn.addEventListener('click', () => {
      const panel = document.getElementById(btn.getAttribute('aria-controls'));
      const open = btn.getAttribute('aria-expanded') === 'true';
      btn.setAttribute('aria-expanded', String(!open));
      panel.hidden = open;
    });
  });
</script>
</body></html>
```

Common mistakes

- Using clickable divs instead of buttons and links.
- Removing the focus outline without a visible replacement.
- Shipping images without width and height, which causes layout shift.
- Testing only with a mouse.

Freelancing opportunities

- Landing page and interface work for product teams and agencies.
- Accessibility audits and fixes for existing sites.
- Freelance briefs for 'HTML/CSS', 'JavaScript', 'React front-end', 'accessibility fix'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- web.dev Learn
- MDN: Learn web development
- WCAG 2.2 quick reference (W3C)

DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 8

E-commerce developer

Profession 8 of 10 in Development and IT.

What the profession means

An e-commerce developer builds the parts of an online shop that handle money and stock: product data, carts, discounts, shipping rules, payments, orders and integrations with warehouses and accounting. Some use hosted platforms (Shopify, WooCommerce); others build custom shops.

What professionals do every day

Models products and prices, writes cart and checkout logic, connects payment and shipping providers, syncs orders and stock, tests money calculations carefully, and monitors failed payments.

Why businesses hire them, and what problems they solve

A shop that gets money calculations wrong loses customers and trust. A developer makes sure totals, discounts, taxes and delivery rules are correct and reliable.

- Wrong totals and discounts at checkout.
- Manual order and stock work between channels.
- Failed or duplicated payments.

Skills you need

- Money handling: store amounts as whole paise or cents, never floating-point errors.
- Carts, discounts and shipping rules.
- Payment gateways and webhooks (for example Razorpay, Stripe).
- Order, stock and accounting integrations.
- Security (PCI scope), testing and monitoring.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Shopify	Hosted e-commerce platform	Trial and development stores	Paid plans	www.shopify.com
WooCommerce	E-commerce plugin for WordPress	Free plugin	Hosting and extensions cost	woocommerce.com
Stripe	Payment processing API	Per-transaction fees (check current pricing)	Per-transaction fees	stripe.com/docs
Razorpay	Payment gateway for India	Test mode	Per-transaction fees	razorpay.com/docs
Node.js test runner	Built-in tests (node:test)	Free	Free	nodejs.org/api/test.html

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Product catalogue and a checkout page; test totals by hand.
Intermediate	Coupons, shipping rules, payment test mode and order emails.
Advanced	Webhooks, stock sync, reporting and a multi-channel integration.

Projects by level

- **Beginner:** A small shop with a cart and one payment method in test mode.
- **Intermediate:** A coupon and shipping engine with automated tests.
- **Advanced:** An integration that syncs orders to accounting software.

Client-ready portfolio project

A checkout for a fictional or permissioned shop: cart, coupon, shipping rules, payment in test mode, order confirmation, and an automated test suite for money calculations.

Worked example

Worked example. A basket of Rs 180 × 2 plus Rs 90 is Rs 450. A 10% coupon takes Rs 45 off, leaving Rs 405. Below the Rs 500 threshold, Rs 49 shipping applies, so the total is Rs 454. The tests in the solution check these numbers.

Assignment with full solution

ASSIGNMENT

Cart totals with coupons and free delivery

Business problem: A bakery's online shop must show the right total before checkout: items, a coupon (percent or flat), and free delivery above a threshold.

Steps

1. Create `cart.js` and `test.js` as shown.
2. Run `node test.js`. All three checks must pass.
3. Add one new test: a flat coupon larger than the subtotal must not give a negative total.
4. Explain the rule you used for the free-delivery threshold, and whether it compares before or after the discount.

Expected output: Passing tests and a one-paragraph explanation of the business rules.

SOLUTION (ANSWER KEY)

Why it works. Each function does one job, so each can be tested alone. Shipping is decided after the discount (the comparison uses `afterDiscount`), which is a business choice the shop must confirm. The flat coupon is capped with `Math.min`, so the discount can never exceed the subtotal: a flat Rs 600 coupon on a Rs 450 basket gives a discount of Rs 450, leaving a subtotal of 0, so

shipping of Rs 49 applies and the total is Rs 49. **Test added:** `assert.strictEqual(cartTotal([{price:100,qty:1}], {type:'flat', value:500}, rule).total, 49)`, which holds because the discount is capped at the subtotal and the Rs 49 fee applies below the threshold. Tax is not included; add it only after checking current rules.

JAVASCRIPT

```
// cart.js : pure functions for a cart total (easy to test)
function lineTotal(item) {
  return item.price * item.qty;
}
function applyCoupon(subtotal, coupon) {
  if (!coupon) return 0;
  if (coupon.type === 'percent') return Math.round(subtotal * coupon.value / 100);
  if (coupon.type === 'flat') return Math.min(coupon.value, subtotal);
  return 0;
}
function cartTotal(items, coupon, shippingRule) {
  const subtotal = items.reduce((sum, item) => sum + lineTotal(item), 0);
  const discount = applyCoupon(subtotal, coupon);
  const afterDiscount = subtotal - discount;
  const shipping = afterDiscount >= shippingRule.freeAbove ? 0 : shippingRule.fee;
  return { subtotal, discount, shipping, total: afterDiscount + shipping };
}
module.exports = { lineTotal, applyCoupon, cartTotal };

// test.js : run with `node test.js`
const assert = require('assert');
const { cartTotal } = require('./cart');
const rule = { freeAbove: 500, fee: 49 };
const basket = [{ price: 180, qty: 2 }, { price: 90, qty: 1 }]; // subtotal 450
assert.deepStrictEqual(cartTotal(basket, null, rule), { subtotal: 450, discount: 0, shipping:
49, total: 499 });
assert.deepStrictEqual(cartTotal(basket, { type: 'percent', value: 10 }, rule),
  { subtotal: 450, discount: 45, shipping: 49, total: 454 });
assert.deepStrictEqual(cartTotal([{ price: 300, qty: 2 }], null, rule),
  { subtotal: 600, discount: 0, shipping: 0, total: 600 });
console.log('all cart tests passed');
```

Common mistakes

- Calculating money with floating-point numbers, which can drift by a paisa.
- Trusting the browser for prices; always recalculate on the server.
- Testing payments in live mode.
- Missing webhook handling, so a paid order never confirms.

Freelancing opportunities

- Shop builds on Shopify and WooCommerce.
- Payment and shipping integrations.
- Freelance briefs for 'checkout', 'Shopify app', 'payment gateway integration', 'order sync'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- Stripe documentation
- Razorpay documentation
- Node.js built-in test runner

DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 9

React.js developer

Profession 9 of 10 in Development and IT.

What the profession means

A React developer builds user interfaces from components: small, reusable pieces that keep their own state and render what the data says. React is the most widely used library for interactive web interfaces, and it is also the usual front end for full-stack apps.

What professionals do every day

Designs components, manages state and data flow, connects to APIs, writes tests, optimises rendering and maintains a component library.

Why businesses hire them, and what problems they solve

Businesses with interactive products (dashboards, carts, booking flows) need interfaces that update smoothly and can grow. React developers build and maintain those interfaces.

- Interfaces that get harder to change as they grow.
- Data that is out of date on screen.
- Slow pages from unnecessary re-renders.

Skills you need

- JavaScript modern syntax (arrow functions, destructuring, modules).
- JSX and components; props and state.
- Hooks (useState, useEffect) and data fetching.
- Routing, forms and testing.
- Performance and deployment.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
React	UI library	Free	Free	react.dev
Vite	Fast development server and build tool	Free	Free	vite.dev
Visual Studio Code	Code editor	Free	Free	code.visualstudio.com
Vercel	Hosting for React apps	Hobby tier	Paid plans	vercel.com
React Developer Tools	Browser extension to inspect components	Free	Free	react.dev/learn/react-developer-tools

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Components, props and state; render lists and handle clicks.
Intermediate	Hooks, fetching data from an API, forms and routing.
Advanced	Testing, performance profiling, context or a state library, and deployment.

Projects by level

- **Beginner:** A counter and a to-do list.
- **Intermediate:** A product list that loads from an API with a search box.
- **Advanced:** A dashboard with routing, charts and login.

Client-ready portfolio project

A React application for a fictional or permissioned business: several pages, data from an API, a form with validation, tests for two components, and deployment with a README.

Worked example

Worked example. A to-do list is an array in state. Adding an item creates a new array with the item at the end; deleting filters the array. React redraws the list because it sees a new array.

Assignment with full solution

ASSIGNMENT

A React counter and a list with a filter

Business problem: A product team wants a small interactive component: a counter that can be reset, and a list of tasks that can be filtered.

Steps

1. Create a project with the official Vite React template: `npm create vite@latest` (follow the current official instructions).
2. Create `Counter.jsx` as in the solution, and render it in `App.jsx`.
3. Add a list component with three tasks, a text filter and a count of visible tasks.
4. Check that each button changes the screen, and that the filter updates the count.

Expected output: A running page with two working components.

SOLUTION (ANSWER KEY)

Why it works. `useState` keeps the value and re-renders the component when it changes. Props (`start`, `step`) make the component reusable. `aria-live` announces changes to screen readers.
Not run here: the solution is syntax-checked as JSX; run it in a Vite project to confirm the page works, and check the current Vite and React documentation for the exact commands.

JAVASCRIPT

```
// Counter.jsx : a React component with state (Vite or Create React App template)
import { useState } from 'react';

export default function Counter({ start = 0, step = 1 }) {
  const [count, setCount] = useState(start);
  return (
    <div>
      <p aria-live="polite">Count: {count}</p>
      <button onClick={() => setCount(count + step)}>Add {step}</button>
      <button onClick={() => setCount(start)}>Reset</button>
    </div>
  );
}

// App.jsx
// import Counter from './Counter';
// export default function App() { return <Counter start={10} step={5} />; }
```

Common mistakes

- Mutating state directly (for example pushing into the array) so the screen does not update.
- Putting every value in global state.
- Calling APIs in render instead of effects or data libraries.
- Missing keys in lists.

Freelancing opportunities

- Interactive dashboards and product pages.
- Component work for existing React codebases.
- Freelance briefs for 'React developer', 'Next.js', 'dashboard', 'frontend React'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- React: Learn (official)
- Vite guide (official)
- Next.js Learn (official)

Bubble.io developer

Profession 10 of 10 in Development and IT.

What the profession means

A Bubble developer builds web applications visually, with no traditional code. Bubble provides a database, page designer, workflows (logic) and privacy rules, so a single builder can deliver a working product.

What professionals do every day

Designs data types and pages, writes workflows for user actions, sets privacy rules for each data type, connects to APIs and payments through plugins, tests as different users and launches the app.

Why businesses hire them, and what problems they solve

Many small businesses and startups need a working prototype or internal tool quickly. A Bubble developer can deliver a booking tool, a directory or a marketplace prototype in weeks rather than months.

- Ideas that need a working product to test with users.
- Internal tools that replace spreadsheets.
- Booking, listing and membership flows.

Skills you need

- Data modelling: types, fields, relationships.
- Workflows and conditions.
- Privacy rules and user roles (the most important security topic in Bubble).
- APIs and plugins; payments.
- Performance, testing as different users, and launch.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Bubble	No-code web app builder	Free plan with limits	Paid plans for custom domains and capacity	bubble.io
Bubble Academy	Official free lessons	Free	Free	bubble.io/academy
Bubble Manual	Official reference	Free	Free	manual.bubble.io

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Pages, data types and forms; one working workflow.

Intermediate	Privacy rules, roles, repeating groups and filters; API connector.
Advanced	Performance, payments, multi-step workflows and launch with a domain.

Projects by level

- **Beginner:** A simple booking form that saves to the database.
- **Intermediate:** A directory with search and filters.
- **Advanced:** A membership site with login and payments in test mode.

Client-ready portfolio project

A booking or directory app for a real or fictional business with user roles, privacy rules, a test log for each role, and a launch checklist.

Worked example

Worked example. A 'Save' button runs the workflow 'Create a new Booking' with the form's values. The data appears in the database tab immediately, which makes testing easy.

Assignment with full solution

ASSIGNMENT

A booking app in Bubble without code

Business problem: A clinic wants a booking app: patients pick a service and a time, the clinic sees the bookings, and only logged-in staff can see the list.

Steps

1. Create a Bubble account and a new app from a blank template (official Bubble Academy lessons are free).
2. Create data types: Service (name, price), Booking (patient name, phone, service, date). Add three Services.
3. Build a booking page with a dropdown of Services, a date picker and a button. Add a workflow: when the button is clicked, create a Booking with the form values.
4. Create a staff page with a repeating group of Bookings. Set privacy rules so only logged-in users with the Staff role can read Bookings.
5. Test as a patient (logged out) and as staff (logged in); confirm that a patient cannot see the staff list.

Expected output: Two pages, a working booking flow, and a privacy-rule test log.

SOLUTION (ANSWER KEY)

Solution outline. The data types hold the information. The workflow 'when button is clicked → create a new Booking' saves the form. The staff page uses a repeating group with the source set to Booking, filtered to the current date if needed. The privacy rule is the important part: without it, anyone with the link can read the list, even if the page is hidden. **Not run here:** Bubble was not

reachable from this session; check the current Bubble Academy steps for privacy rules and workflow names before following them.

Common mistakes

- Leaving the privacy rules open, so private data is visible through the API.
- Building every page before testing one workflow end to end.
- Using many plugins without checking their costs and limits.
- Not testing as different user roles.

Freelancing opportunities

- Prototypes and MVPs for startups.
- Internal tools and booking flows for small businesses.
- Freelance briefs for 'Bubble', 'no-code app', 'MVP', 'booking app'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- Bubble Academy (official)
- Bubble Manual (official)
- Bubble Forum (official community)

PART 3 · MODULE D

Advanced development: back end, security, performance, testing and deployment

The advanced topics that turn a demo into a product a business can depend on.

These topics apply across the ten professions. Read each section, then use one of them in a project from the chapters above.

D1. Backend development and APIs

The back end receives requests, checks them, applies business rules, reads or writes the database and returns a response. Keep endpoints small and named after the resource (`/api/bookings`). Return the right status code (see Part 1, Module A).

D2. Authentication and authorisation

Authentication proves who the user is; **authorisation** decides what they may do. Use a tested service (for example Supabase Auth, Firebase Auth or a framework's built-in library) rather than writing your own. Store passwords only as hashes. Check permissions on the server for every request.

D3. Database management

Design tables around the questions the app asks. Add indexes to columns you search often. Back up regularly and test a restore. Never give the public app the database administrator key.

D4. API integration

Read the provider's documentation, store keys in environment variables, handle timeouts and errors, and respect rate limits. Use webhooks for events that happen later (for example a payment confirmed).

D5. Payment gateways

Use the provider's hosted checkout or tested SDK; never store card numbers yourself. Confirm payment with a server-side webhook, not only the browser redirect. Test in the provider's test mode first (see the E-commerce chapter).

D6. Website security

- Use HTTPS everywhere and keep software updated.
- Validate input on the server; escape output to prevent script injection.
- Set secure cookies and a content security policy where the platform allows.
- Keep secrets out of code and out of public repositories.

D7. Performance optimisation

Measure first (Lighthouse, browser performance panel). Compress and size images, load scripts only where needed, cache static files, and avoid unnecessary database queries (for example, load a list once instead of once per row).

D8. Testing

Test the rules that matter most: money calculations, permissions, form validation. The Node.js built-in test runner, a browser test tool (Playwright) and manual checks on a phone cover most small projects.

D9. Deployment, cloud hosting and maintenance

Deploy from git, with a staging copy for testing. Use managed hosting for static sites and serverless functions (Vercel, Netlify, GitHub Pages), and a managed database. Monitor errors, renew domains and certificates, and update dependencies on a schedule. Write a one-page handover for the client.

PART 4 · MODULE E

Freelancing for developers: packages, discovery, proposals, pricing and delivery

How to sell development work once you can build it.

E1. Service packages you can offer

Package the work so the client can compare options. Prices depend on your region, experience and the client's market; no price here is a market rate.

Package	What the client gets	Illustrative scope (not market data)
Landing page	One responsive page, contact form, basic SEO, analytics	A single page with 3 to 5 sections
Business website	Up to 5 pages, CMS or easy editing, performance and accessibility checks	Brochure site with a contact and enquiry form
E-commerce store	Store setup, product import, payment test and live, shipping and tax set-up checked against official rules	Shopify or WooCommerce store with a product list
Maintenance	Monthly updates, backups, security checks, small edits, a monthly report	Ongoing service billed monthly
Custom application	Requirements, design, build, testing, deployment, handover	Booking, portal or dashboard project priced by milestone

E2. Client discovery

Ask about goals, users, content, deadlines and success measures before you quote. Write the answers down; they become your scope.

TEXT

```
Discovery call script (20 minutes)
1. Open: 'Thanks for your time. I will ask a few questions, then suggest what fits.'
2. Goal: 'What must the website achieve in the next 3 months?'
3. Users: 'Who visits, and what do they need to do?'
4. Content: 'What do you already have: text, photos, logo?'
5. Constraints: 'What is your deadline and budget range?'
6. Success: 'How will we know it worked?'
7. Close: 'I will send a short proposal by [day]. Shall we confirm the next step?'
```

E3. Proposal writing

A good proposal states the outcome, what is included, what is not, the timeline, the price, payment terms and the assumptions. A complete example:

TEXT

```
Subject: Proposal: responsive landing page for Asha's Bakery

Hello Asha,

Thank you for the brief. Here is what I propose.
```

```
What I will build
- A one-page responsive site with your menu, opening hours and an order form.
- Form checks for phone numbers, and an email to you when an order arrives.
- Accessibility and speed checks on mobile and desktop, with the results shared.

Timeline
- Day 1-2: confirm content and photos.
- Day 3-5: design and build, with one preview link for your feedback.
- Day 6: final changes (two rounds of revisions are included), launch and handover.

Price (fixed)
- Rs [amount], including hosting for the first year as set out in the agreement.
- Payment: 50% to start, 50% on launch.

Assumptions: you provide the text and photos; any third-party costs are shown before you pay.

I can start on [date] once the deposit is received. Happy to answer any questions.

[Your name] | [phone] | [portfolio link]
```

E4. Pricing

Three simple methods, with example arithmetic (illustrative numbers only):

- **Hourly:** estimated hours × hourly rate. Example: 30 hours × Rs 800 = Rs 24,000 (illustrative).
- **Fixed:** price the scope, add a buffer for revisions. Example: Rs 24,000 scope + 15% revision buffer = Rs 27,600.
- **Retainer:** a monthly fee for a defined amount of work, for maintenance. Example: 4 hours a month at Rs 800 plus hosting, illustrative only.

Include software and hosting costs in the quote, and say which subscriptions the client pays for directly. Do not state earnings or income guarantees to clients or students.

E5. Delivery and handover

Deliver to a staging link, get written approval, launch, then hand over: login details (through a password manager, never plain text), a one-page guide, backup and update notes, and the support period.

E6. Post-project support

Offer a fixed support window (for example 14 days for bugs) and a paid maintenance plan after it. Record every change request in writing.

APPENDIX

Resource directory (Development and IT)

Official documentation, training portals and marketplaces, with the cost status of each.

Every link below is an official or widely used source. None was checked live in this session, because most of these domains were blocked from the research environment. Confirm each link and each price before publishing. No certification is claimed here: the resources teach skills; they do not confer accreditation.

Resource	Link	Profession or topic	Free or paid
MDN Web Docs: Learn web development	developer.mozilla.org/en-US/docs/Learn_web_development	Web basics (all)	Free
The Odin Project	www.theodinproject.com	Full-stack, front-end	Free
Git (Pro Git book)	git-scm.com/book/en/v2	All professions	Free
GitHub Skills	github.com/skills	Git and GitHub	Free
Node.js learning resources	nodejs.org/en/learn	Full-stack, back end	Free
Node.js test runner	nodejs.org/api/test.html	E-commerce, testing	Free
Supabase documentation	supabase.com/docs	Full-stack, database, auth	Free docs; paid plans
WordPress documentation	wordpress.org/documentation/	WordPress	Free
WordPress Learn (official)	learn.wordpress.org	WordPress	Free
WordPress developer handbook: child themes	developer.wordpress.org/themes/advanced-topics/child-themes/	WordPress	Free
web.dev Learn (Google)	web.dev/learn	Web, front-end, performance	Free
W3C WAI accessibility tutorials	www.w3.org/WAI/tutorials/	Front-end, web	Free
WCAG 2.2 quick reference	www.w3.org/WAI/WCAG22/quickref/	Front-end, accessibility	Free
W3C HTML validator	validator.w3.org	Web	Free
Shopify Dev Docs	shopify.dev	Shopify	Free
Shopify Help Centre	help.shopify.com	Shopify	Free
Shopify Partners programme	www.shopify.com/partners	Shopify developers	Free to join
Webflow University	university.webflow.com	Webflow	Free
Bubble Academy	bubble.io/academy	Bubble.io	Free
Bubble Manual	manual.bubble.io	Bubble.io	Free
React: Learn (official)	react.dev/learn	React.js	Free
Vite guide (official)	vite.dev/guide/	React.js, build tools	Free

Next.js Learn (official)	nextjs.org/learn	React, full-stack	Free
Expo documentation	docs.expo.dev	Mobile app	Free
React Native: getting started	reactnative.dev/docs/getting-started	Mobile app	Free
Flutter documentation	docs.flutter.dev	Mobile app	Free
Android Developers courses	developer.android.com/courses	Mobile app	Free
Stripe documentation	stripe.com/docs	E-commerce, payments	Free docs; per-transaction fees
Razorpay documentation	razorpay.com/docs	E-commerce, payments (India)	Free docs; per-transaction fees
Visual Studio Code	code.visualstudio.com	All professions	Free
Chrome DevTools documentation	developer.chrome.com/docs/devtools	Front-end, web	Free
Lighthouse	developer.chrome.com/docs/lighthouse	Performance and accessibility	Free
Upwork (freelance marketplace)	www.upwork.com	Finding work and skill-based projects	Free to join; fees apply
Fiverr (freelance marketplace)	www.fiverr.com	Packaged services	Free to join; fees apply
Freelancer.com	www.freelancer.com	Project bids	Free to join; fees apply
Netlify	www.netlify.com	Hosting	Free tier
Vercel	vercel.com	Hosting	Free tier
GitHub Pages	pages.github.com	Free hosting	Free